

Wallet loaded

Address A': 2N8cMS9qHVjeACq9rtDMTZkkawxT5F2gT6m

Address B': 2MvrVvY6gBQiUFmtVgSxWLeysQTtciBaVBJ

Address C': 2NFbMYv5xSZLyqJb9Yvat1paLup16pqBAyU

Funding TX: 8d82c0042ed89877c7cf12730d9f889e7a821bf5dcfb65b179a2e1c1d1f5bb30

UTXO A': 8d82c0042ed89877c7cf12730d9f889e7a821bf5dcfb65b179a2e1c1d1f5bb30

Raw TX A' -> B':

0200000000130bbf5d1c1e1a279b165fbdcf51b827a9e889f0d7312cfc77798d82e04c0828d0000000
0000fdffffff016043993b0000000017a91427943cfb90faca27a0ccc3377e9bf9e7d76d861d8700
0000000

Signed TX 1:

```
{
  'hex':
    '02000000000010130bbf5d1c1e1a279b165fbdcf51b827a9e889f0d7312cfc77798d82e04c0828d0
    000000001716001405f6c8a6582e8af0553ca293e509a494266b6b43fdffffff016043993b00000000
    017a91427943cfb90faca27a0ccc3377e9bf9e7d76d861d870247304402200a8a0c78199c6631f7e
    c8e19651592dbacfa1bce6315216a437a9e7eb21b38cd02201ff5d42f76ee7383076787a84974471
    90018adfd0bfe7a973f512e03e9c59ed6012102dd0285f6ed2428afa13cde88ee8f98b0733c50a53
    1960fd9c05b5117e9aee10a00000000',
  'complete': True
}
```

Decoded Transaction:

```
{
  'txid': '9404fe308d80043a734bdc00f3ed9ca64393a18def88cdd841577373e147f657',
  'hash': '5f565adf8020e52646ea94b9a74f94bf943ffd4f0872e31c5bf04f516f22e554',
  'version': 2,
  'size': 215,
  'vsize': 134,
  'weight': 533,
  'locktime': 0,
  'vin':
    [
      {
        'txid':
          '8d82c0042ed89877c7cf12730d9f889e7a821bf5dcfb65b179a2e1c1d1f5bb30',
        'vout': 0,
        'scriptSig':
          {
            'asm': '001405f6c8a6582e8af0553ca293e509a494266b6b43',
            'hex': '16001405f6c8a6582e8af0553ca293e509a494266b6b43'
          },
        'txinwitness':
          [
            '304402200a8a0c78199c6631f7ec8e19651592dbacfa1bce6315216a437a9e7eb21b38cd02201ff
            5d42f76ee7383076787a8497447190018adfd0bfe7a973f512e03e9c59ed601',
            '02dd0285f6ed2428afa13cde88ee8f98b0733c50a531960fd9c05b5117e9aee10a'
          ],
            'sequence': 4294967293
          }
    ],
  'vout':
    [
      {
        'value': Decimal('9.99900000'),
        'n': 0,
        'scriptPubKey':
          {
            'asm': 'OP_HASH160 27943cfb90faca27a0ccc3377e9bf9e7d76d861d
            OP_EQUAL',
            'desc': 'addr(2MvrVvY6gBQiUFmtVgSxWLeysQTtciBaVBJ)#prpj8sns',
            'hex': 'a91427943cfb90faca27a0ccc3377e9bf9e7d76d861d87',
            'address': '2MvrVvY6gBQiUFmtVgSxWLeysQTtciBaVBJ',

```

```

        'type': 'scripthash'
    }
}
]
}
Locking Script (ScriptPubKey):
{
    'asm': 'OP_HASH160 27943cfb90faca27a0ccc3377e9bf9e7d76d861d OP_EQUAL',
    'desc': 'addr(2MvrVvY6gBQiUFmtVgSxWLeysQTtciBaVBJ)#prpj8sns',
    'hex': 'a91427943cfb90faca27a0ccc3377e9bf9e7d76d861d87',
    'address': '2MvrVvY6gBQiUFmtVgSxWLeysQTtciBaVBJ',
    'type': 'scripthash'
}
TXID A'->B': 9404fe308d80043a734bdc00f3ed9ca64393a18def88cdd841577373e147f657
Raw TX B'->C':
0200000000157f647e173735741d8cd88ef8da19343a69cedf300dc4b733a04808d30fe0494000000
0000fdfffff01c0bc973b0000000017a914f52166419bca0d1be51e6564f9c71dd0f239f2768700
000000
Signed TX 2:
{
    'hex':
'02000000000010157f647e173735741d8cd88ef8da19343a69cedf300dc4b733a04808d30fe04940
00000001716001489dcc378331c1b2bec66ccaa187c654f2d181b0cfdfffff01c0bc973b00000000
017a914f52166419bca0d1be51e6564f9c71dd0f239f2768702473044022055b68b5021fbd9fb857
3af56e9f370edec79ee748a7d67f1641bfc5c9b2d557702200b8102e9f17d67032e73adc70f255e4
8479465b906f245ee6377e9553ef5855c0121025d1de9c5e6bbc4a904acc4569bd2284f7356c7182
08f88ea5b40bf3a1d41085c00000000',
    'complete': True
}
Decoded B'->C':
{
    'txid': 'd135761de373fe27b41d9a7f3f83c7714180106e02454403cdd41155573ee49f',
    'hash': 'f2a0f788792c00750793c1e6f3ae08b77789586b2d1befe681aabfe7354fa34c',
    'version': 2,
    'size': 215,
    'vsize': 134,
    'weight': 533,
    'locktime': 0,
    'vin':
    [
        {
            'txid':
'9404fe308d80043a734bdc00f3ed9ca64393a18def88cdd841577373e147f657',
            'vout': 0,
            'scriptSig':
            {
                'asm': '001489dcc378331c1b2bec66ccaa187c654f2d181b0c',
                'hex': '16001489dcc378331c1b2bec66ccaa187c654f2d181b0c'
            },
            'txinwitness':
            [
                '3044022055b68b5021fbd9fb8573af56e9f370edec79ee748a7d67f1641bfc5c9b2d557702200b8
102e9f17d67032e73adc70f255e48479465b906f245ee6377e9553ef5855c01',
                '025d1de9c5e6bbc4a904acc4569bd2284f7356c718208f88ea5b40bf3a1d41085c'
            ],
            'sequence': 4294967293
        }
    ],
    'vout':
    [
        {
            'value': Decimal('9.99800000'),

```

```

        'n': 0,
        'scriptPubKey':
        {
            'asm': 'OP_HASH160 f52166419bca0d1be51e6564f9c71dd0f239f276
OP_EQUAL',
            'desc': 'addr(2NFbMYv5xSZLyqJb9Yvat1paLup16pqBAYU)#zcejhnfk',
            'hex': 'a914f52166419bca0d1be51e6564f9c71dd0f239f27687',
            'address': '2NFbMYv5xSZLyqJb9Yvat1paLup16pqBAYU',
            'type': 'scripthash'
        }
    ]
}
scriptSig:
{
    'asm': '001489dcc378331c1b2bec66ccaa187c654f2d181b0c',
    'hex': '16001489dcc378331c1b2bec66ccaa187c654f2d181b0c'
}
Witness Data:
[
    '3044022055b68b5021fbd9fb8573af56e9f370edec79ee748a7d67f1641bfc5c9b2d557702200b8
102e9f17d67032e73adc70f255e48479465b906f245ee6377e9553ef5855c01',
    '025d1de9c5e6bbc4a904acc4569bd2284f7356c718208f88ea5b40bf3a1d41085c'
]
TXID B'->C': d135761de373fe27b41d9a7f3f83c7714180106e02454403cdd41155573ee49f

```

I tried using btcdeb (bitcoin debugger) to test the script is correct or not but could not find the correct syntax for it. I was running the below command.

```

./btcdeb
3044022055b68b5021fbd9fb8573af56e9f370edec79ee748a7d67f1641bfc5c9b2d557702200b81
02e9f17d67032e73adc70f255e48479465b906f245ee6377e9553ef5855c01
025d1de9c5e6bbc4a904acc4569bd2284f7356c718208f88ea5b40bf3a1d41085c OP_DUP
OP_HASH160 89dcc378331c1b2bec66ccaa187c654f2d181b0c OP_EQUALVERIFY OP_CHECKSIG
but it kept giving the output as invalid script.
When I ran the same script on an online checker, it gave output as correct. A
video of the online checker has been attached to refer. The online website used
is https://wschae.github.io/build/editor.html.

```

UNDERSTANDING THE SEGWIT TRANSACTION IN DETAIL

We take the example of the signed transaction B' → C'. Below is the hex code of the signed transaction:

```

02000000000010157f647e173735741d8cd88ef8da19343a69cedf300dc4b733a04808d30fe049400
0000001716001489dcc378331c1b2bec66ccaa187c654f2d181b0cfdffffff01c0bc973b00000000
17a914f52166419bca0d1be51e6564f9c71dd0f239f2768702473044022055b68b5021fbd9fb8573
af56e9f370edec79ee748a7d67f1641bfc5c9b2d557702200b8102e9f17d67032e73adc70f255e48
479465b906f245ee6377e9553ef5855c0121025d1de9c5e6bbc4a904acc4569bd2284f7356c71820
8f88ea5b40bf3a1d41085c00000000
02000000 0001 01
57f647e173735741d8cd88ef8da19343a69cedf300dc4b733a04808d30fe0494 00000000 17
16001489dcc378331c1b2bec66ccaa187c654f2d181b0c fdffffff 01 c0bc973b00000000 17
a914f52166419bca0d1be51e6564f9c71dd0f239f27687
02473044022055b68b5021fbd9fb8573af56e9f370edec79ee748a7d67f1641bfc5c9b2d5577
02200b8102e9f17d67032e73adc70f255e48479465b906f245ee6377e9553ef5855c01 21
025d1de9c5e6bbc4a904acc4569bd2284f7356c718208f88ea5b40bf3a1d41085c 00000000

```

The first 4 bytes are the version number
Version: 02000000

The next two bytes indicate that the transaction is using SegWit.
marker: 00
flag: 01

The next byte is the input counter
Input Counter: 01

The next 32 bytes are the hash of tx_id of the previous transaction or the input transaction being used
Hash of the spent transaction:
57f647e173735741d8cd88ef8da19343a69cedf300dc4b733a04808d30fe0494

The next 4 bytes represent the index of the output in the output of previous transaction
TxOut index: 00000000 --> Means the first output of the previous transaction

The next byte represents the length of script
Script length: 17 --> $1 \times 16 + 7 = 23$ Bytes

The next 23 bytes represent the input script
Input Script: 16001489dcc378331c1b2bec66ccaa187c654f2d181b0c
16 00 14 89dcc378331c1b2bec66ccaa187c654f2d181b0c

Dissection of the input script
16
00 --> Witness version (SegWit v0)
14 --> push 20 bytes (40 characters)
Next 20 bytes (40 characters): pubkey_hash =
89dcc378331c1b2bec66ccaa187c654f2d181b0c

Hence the unlocking script is 0 <pubKeyHash>

The next 4 bytes represent the sequence
Sequence: fdfdfdfdf

The next bytes are the output
Output: 01c0bc973b0000000017a914f52166419bca0d1be51e6564f9c71dd0f239f27687
01 c0bc973b00000000 17 a914f52166419bca0d1be51e6564f9c71dd0f239f27687

The first byte is the number of outputs:
Number of outputs: 01

The next 8 bytes are the amount being sent
Amount: c0bc973b00000000 --> 3b97bcc0 --> hex2dec: 999800000 Satoshis

The next byte is the length of the output script:
Length: 17 --> hex2dec --> $1 \times 16 + 7 = 23$ Bytes = 46 characters
Output Script(scriptPubKey): a9 14 f52166419bca0d1be51e6564f9c71dd0f239f276 87
a9 --> OP_HASH160
14 --> push 20 bytes (40 characters)
f52166419bca0d1be51e6564f9c71dd0f239f276 --> script hash
87 --> OP_EQUAL

Hence the locking script is OP_HASH160 <scriptHash> OP_EQUAL

Unlike legacy transactions, the unlocking data is stored in the witness section.
Witness data: 02 47
3044022055b68b5021fbd9fb8573af56e9f370edec79ee748a7d67f1641bfc5c9b2d557702200b81
02e9f17d67032e73adc70f255e48479465b906f245ee6377e9553ef5855c01 21
025d1de9c5e6bbc4a904acc4569bd2284f7356c718208f88ea5b40bf3a1d41085c 00000000

```
02 --> 2 witness stack elements
47 --> hex2dec --> 71 Bytes = 142 characters (length of 1st witness item)

Signature:
3044022055b68b5021fbd9fb8573af56e9f370edec79ee748a7d67f1641bfc5c9b2d557702200b81
02e9f17d67032e73adc70f255e48479465b906f245ee6377e9553ef5855c01

21 --> hex2dec --> 33 Bytes = 66 characters (length of 2nd witness item)

Public Key: 025d1de9c5e6bbc4a904acc4569bd2284f7356c718208f88ea5b40bf3a1d41085c

00000000: Transaction locktime
```

We understand from this one complete hash of transaction B'→C' that it contains all the required information for the given transaction. It starts with the version number followed by the hash of the input being used. This hash is used to refer to the input transaction and hence the locking script that was associated with it.

The next bytes represent the input script or the unlocking script that contain pubkey_hash that is used against the locking script to prove that this transaction can use the previous UTXO.

The previous UTXO is destroyed and new UTXO is created. Since there is no change left (i.e. all the amount of the previous UTXO is being sent) only one output exists.

A new locking script is created with this transaction that contains the hash of the public key of user C'. When C' wants to spend this UTXO, this locking script will be checked against the signature and the public key provided by the user C'.

To check that the UTXO being spent belongs to the spender, the locking script follows the below procedure.

For P2SH-P2WPKH transactions, validation occurs in multiple steps. The scriptSig pushes the unlocking script: 0 <pubKeyHash>. The system checks:

`HASH160(unlockingScript) == scriptHash`. If true, the unlocking script is accepted.

The witness then provides: <signature> <public key>

The unlocking script expands to:

`OP_DUP OP_HASH160 <pubKeyHash> OP_EQUALVERIFY OP_CHECKSIG`

`<signature> <pubkey> OP_DUP OP_HASH160 <pubkey_hash> OP_EQUALVERIFY OP_CHECKSIG`

This is a stack based algorithm. The user provides their signature and public key which are pushed into the stack.

The script duplicates the <pubkey> present at the top of the hash and pushes it into the hash.

The already present <pubkey_hash> in the script is then pushed into the stack.

The `OP_EQUALVERIFY` function then pops out the top two elements and compares them. If they are equal then the script proceeds further otherwise stops.

The `OP_CHECKSIG` then pops out the <pubkey> and <signature> present in the stack and verifies that the signature belongs to the provided pubkey only. If true a 1 is pushed into the stack. This marks the end of the algorithm.

This is the same validation logic as P2PKH, but the unlocking data comes from the witness field. If valid, the script returns TRUE.

The full process is that we created a wallet named testwallet under which we created 3 P2SH-SegWit addresses (A', B', C'). We first send 10 bitcoins to A (since we are in regtest we can directly send bitcoins using the function `sendtoaddress(<address>, <number_of_bitcoins>)`).

We then get the utxo set and find the utxo that is owned by A'. This utxo has information of owner address, amount and the locking script and witness data which must be unlocked using the signature and the pubkey of the owner.

This utxo is then used as an input to send the bitcoins to B'. A fee of 0.001 BTC is charged and a new locking script is created with the pubkey_hash of B.

The previous UTXO is then destroyed and a new UTXO is created that has the address of B.

This new UTXO is then used in a new transaction of sending the bitcoins (now 9.999 BTC) to C' with a fee charge of 0.001 BTC. The locking and unlocking are done again using the signature and pubkey of B' and pubkey_hash of C' respectively.

The new UTXO generated is now addressed to C' and can be used for a new transaction when the signature and pubkey are provided.